Лекция 3. Методы предотвращения угроз надёжности. Оперативные методы повышения надёжности: временная, информационная, программная избыточность.

Введение

В предыдущей лекции мы подробно разобрали, что такое объекты уязвимости, дестабилизирующие факторы и угрозы надёжности. Мы выяснили, что любая компьютерная система потенциально уязвима, и что угрозы могут реализоваться через конкретные слабые места под воздействием различных факторов — как внешних, так и внутренних, как случайных, так и преднамеренных.

Однако знание угроз — это только половина дела. Главный вопрос для специалиста по обеспечению качества функционирования компьютерных систем звучит так: что можно сделать, чтобы предотвратить реализацию этих угроз или минимизировать их последствия?

Ответ на этот вопрос лежит в области методов обеспечения надёжности. Все эти методы можно разделить на две большие группы:

1. Профилактические (предупредительные) методы — направлены на то, чтобы угроза вообще не реализовалась. Это борьба с причинами.

2. Оперативные методы — направлены на то, чтобы система сохранила работоспособность даже при возникновении отказа или сбоя. Это борьба с последствиями.

В данной лекции мы сосредоточимся именно на оперативных методах, и главным инструментом здесь является **избыточность**. Мы разберём три её вида: временную, информационную и программную.

1. Общая концепция избыточности

1.1. Что такое избыточность

Избыточность (redundancy) — это наличие в системе дополнительных ресурсов, элементов или возможностей, которые превышают минимально необходимые для выполнения заданных функций в нормальных условиях.

Простыми словами, избыточность — это «запас». Если для решения задачи достаточно одного процессора, а мы ставим два — это избыточность. Если для передачи данных достаточно восьми бит, а мы передаём двенадцать — это тоже избыточность.

1.2. Почему избыточность работает

Основной принцип избыточности прост: если один элемент отказывает, его функцию берёт на себя другой. Именно поэтому избыточность является фундаментом всех методов обеспечения надёжности и отказоустойчивости.

В теории надёжности различают несколько **видов избыточности**:

- структурная (аппаратная) — дублирование оборудования;

- функциональная — дублирование функций;

- временная — использование дополнительного времени;

- информационная — использование дополнительных данных;

- программная — использование дополнительных программных модулей.

2. Временная избыточность

Временная избыточность заключается в том, что система выполняет одну и ту же задачу несколько раз за определённый промежуток времени, а результаты этих выполнений сравниваются для выявления возможных ошибок.

Идея проста: если операция выполнена трижды и все три результата совпадают, вероятность ошибки крайне мала. Если результаты различаются — система может либо повторить операцию, либо предпринять корректирующие действия.

Временная избыточность может реализовываться несколькими способами:

Многократное выполнение операции. Система выполняет одну и ту же операцию несколько раз (обычно нечётное количество — три или пять) и сравнивает результаты. Совпадающие результаты считаются правильными.

Отложенная проверка. Результаты операции проверяются не сразу, а через некоторый интервал времени. Это полезно в системах мониторинга, где повторные измерения могут подтвердить или опровергнуть предыдущие данные.

Контрольные точки (checkpoints). Система периодически сохраняет своё текущее состояние. В случае сбоя она может вернуться к последней сохранённой контрольной точке и повторить вычисления с этого момента.

Примеры применения:

Астрономические и научные вычисления. В системах, где точность критически важна, одна и та же операция может выполняться несколько раз с последующим сравнением результатов.

Системы реального времени. В системах мониторинга повторные измерения с интервалами позволяют отфильтровать случайные выбросы и помехи.

Восстановление после сбоев. Механизм контрольных точек широко используется в базах данных и вычислительных системах для минимизации потерь при отказах.

3. Информационная избыточность

Информационная избыточность заключается в добавлении к данным дополнительной информации, которая позволяет обнаружить или исправить ошибки, возникшие при хранении или передаче этих данных.

Основная идея: к полезным данным добавляются избыточные биты, которые вычисляются по определённому алгоритму. При чтении или приёме данных эти избыточные биты проверяются. Если они не соответствуют данным — значит, произошла ошибка.

Различают два типа информационной избыточности:

Обнаруживающие коды (error-detecting codes). Позволяют только обнаружить факт ошибки. Пример — бит чётности (parity bit). Если ошибка обнаружена, система может запросить повторную передачу данных.

Исправляющие коды (error-correcting codes). Позволяют не только обнаружить, но и исправить ошибку. Пример — ECC-память (Error-Correcting Code memory), которая автоматически исправляет однобитные ошибки.

Примеры применения:

Оперативная память. ECC-память использует 72 бита для хранения 64 бит данных. Дополнительные 8 бит позволяют исправить однобитную ошибку и обнаружить двухбитную.

Жёсткие диски. Данные на дисках защищаются с помощью кодов, обнаруживающих ошибки, а также резервных секторов. Если сектор повреждён, его содержимое может быть восстановлено.

Сетевые протоколы. При передаче данных по сети используются контрольные суммы (например, в TCP/IP), позволяющие обнаружить повреждение пакетов.

Базы данных. Транзакции защищаются с помощью журналирования и контрольных сумм, что позволяет восстановить целостность данных после сбоя.

4. Программная избыточность

Программная избыточность — это использование нескольких независимых версий программного модуля, выполняющих одну и ту же функцию, с последующим сравнением результатов.

Ключевое слово здесь — независимых. Если две версии программы написаны одним и тем же программистом по одному и тому же алгоритму, они, скорее всего, будут содержать одни и те же ошибки. Если же версии разработаны разными группами программистов, с использованием разных инструментов и разных подходов — вероятность совпадения ошибок резко снижается.

Принцип работы:

Программная избыточность реализуется через мультиверсионное программирование (N-version programming). Суть метода:

1. Одна и та же задача реализуется N независимыми версиями программного модуля.

2. Все версии получают одинаковые входные данные.

3. Результаты всех версий сравниваются с помощью решающего органа (voter).

4. Если результаты совпадают — они считаются правильными.

5. Если результаты различаются — система может либо запросить повторное выполнение, либо использовать мажоритарный принцип (выбрать результат, который дала большинство версий).

Существует несколько подходов к добавлению программной избыточности:

N-версное программирование. N независимых версий одного модуля. Результат определяется голосованием.

Модифицированное дуальное программирование. Две версии, результаты сравниваются с учётом допустимой погрешности. Если разница превышает порог — выбирается результат резервной версии.

Контрольные функции. Использование взаимосвязанных функций (например, sin(x) и cos(x)), которые позволяют не только обнаружить ошибку, но и восстановить результат.

Мажоритарный принцип (TMR — Triple Modular Redundancy). Три версии, результат определяется большинством голосов. Это наиболее распространённый вариант.

5. Структурная (аппаратная) избыточность

Структурная избыточность (её также называют аппаратной или схемной) — это вид избыточности, при котором в систему вводятся дополнительные аппаратные элементы, дублирующие или резервирующие функции основных элементов.

Иными словами, если для выполнения задачи достаточно одного процессора, одного блока питания или одного сетевого кабеля, то при структурной избыточности ставят два, три или более таких элементов. Если один отказывает — его функцию берёт на себя другой.

Структурная избыточность реализуется через различные схемы резервирования. Рассмотрим основные.

Общее резервирование

При общем резервировании резервируется вся система целиком. Основная система и одна или несколько резервных работают параллельно или находятся в режиме ожидания. Если основная система отказывает — включается резервная.

Пример: два сервера, один из которых основной, а второй — резервный. При отказе основного резервный берёт на себя все функции.

Раздельное (поэлементное) резервирование

При раздельном резервировании резервируются отдельные элементы системы, а не вся система целиком. Например, отдельно резервируются процессор, блок питания, жёсткий диск.

Пример: сервер с двумя блоками питания, двумя процессорами, RAID-массивом дисков.

Смешанное резервирование

Смешанное резервирование — комбинация общего и раздельного. Часть системы резервируется целиком, часть — поэлементно. Это позволяет оптимизировать затраты и надёжность.

Пример: критически важный узел дублируется целиком, а остальные элементы — по отдельности.

Структурная избыточность может работать в разных режимах:

Нагруженный резерв (hot standby). Резервный элемент работает параллельно с основным и выполняет ту же нагрузку. При отказе основного — резервный сразу берёт управление.

Облегчённый резерв (warm standby). Резервный элемент находится в рабочем состоянии, но не выполняет нагрузку. При отказе основного — включается в работу.

Ненагруженный резерв (cold standby). Резервный элемент выключен и включается только при отказе основного.

6. Функциональная избыточность

Функциональная избыточность — это вид избыточности, при котором система способна выполнять дополнительные функции, не требуемые для нормальной работы, но позволяющие перераспределить нагрузку или компенсировать отказ других компонентов.

В отличие от структурной избыточности, где дублируются элементы, здесь дублируются или расширяются функции. Система может выполнять одну и ту же задачу разными способами, используя разные компоненты.

Принцип работы:

Основная идея: если один компонент отказывает, его функцию может взять на себя другой компонент, изначально предназначенный для других задач, но способный выполнить и эту.

Пример: в многопроцессорной системе при отказе одного процессора его задачи перераспределяются между оставшимися. Каждый процессор изначально предназначен для своей задачи, но может выполнить и чужую.

Основные подходы:

Перераспределение нагрузки. Система динамически перераспределяет задачи между компонентами в зависимости от их доступности и загруженности.

Многофункциональность компонентов. Каждый компонент способен выполнять несколько функций, что позволяет гибко переключаться при отказе.

Функциональное дублирование. Одна и та же функция может быть реализована разными компонентами.

Адаптивная реконфигурация. Система автоматически перестраивает свою структуру при отказе элемента.

7. Комплексное применение всех видов избыточности

На практике для достижения максимальной надёжности виды избыточности применяются совместно. Рассмотрим пример — система онлайн-банкинга:

- Структурная избыточность: два сервера приложений, два блока питания, RAID-массив дисков, резервный канал связи.

- Информационная избыточность: ECC-память, контрольные суммы транзакций, журналирование.

- Программная избыточность: независимые версии модуля расчёта процентов.

- Временная избыточность: контрольные точки при обработке транзакций, повторные проверки.

- Функциональная избыточность: при отказе одного сервера его нагрузка перераспределяется между оставшимися.

Такое сочетание обеспечивает высокий уровень надёжности и отказоустойчивости.

10. Практическое задание

Задание: «Комплексное применение видов избыточности»

- Вариант А: Система управления беспилотным автомобилем.

- Вариант Б: Сервер онлайн-банкинга.

- Вариант В: Бортовой компьютер космического аппарата.

Ход выполнения:

Шаг 1. Определите 5 критических функций выбранной системы, отказ которых недопустим.

Шаг 2. Для каждой критической функции предложите вид избыточности (или комбинацию видов), который позволит предотвратить или минимизировать последствия отказа. Обоснуйте выбор.

Шаг 3. Заполните таблицу:

Критическая функция	Вид угрозы	Вид избыточности	Конкретная реализация	Обоснование

ПРИМЕР:

Краткое описание системы

Система умного дома — это комплекс аппаратных и программных средств, которые автоматизируют управление освещением, отоплением, вентиляцией, безопасностью и бытовыми приборами. Система включает:

- центральный контроллер (хаб);
- датчики (движения, температуры, влажности, дыма, протечки);
- исполнительные устройства (реле, замки, клапаны, лампы);
- мобильное приложение для управления;
- облачный сервис для удалённого доступа;
- локальную сеть.

Система работает круглосуточно, управляет критичными функциями (безопасность, отопление), должна реагировать на события в реальном времени.

Критическая функция	Вид угрозы	Вид избыточности	Конкретная реализация	Обоснование
Обнаружение дыма	Отказ датчика	Структурная	Два независимых датчика дыма в каждой комнате	Если один датчик откажет второй продолжит monitoring
Обнаружение дыма	Сбой передачи сигнала	Информационная	Контрольные суммы и подтверждение доставки сообщений	Гарантирует что сигнал тревоги не потеряется
Обнаружение дыма	Ошибка в прошивке	Программная	Две независимые версии модуля анализа данных	Снижает риск ложного срабатывания или пропуска
Обнаружение протечки	Отказ датчика	Структурная	Датчики протечки в нескольких точках плюс резервный датчик	Повышает вероятность обнаружения
Обнаружение протечки	Залипание клапана	Функциональная	Дублирующий клапан с независимым приводом	Если один клапан не закроется второй перекроет воду
Обнаружение протечки	Потеря данных о событии	Информационная	Журналирование событий с контрольными суммами	Позволяет восстановить картину после сбоя
Управление отоплением	Отказ контроллера	Структурная	Резервный контроллер в режиме hot standby	Мгновенное переключение при отказе основного
Управление отоплением	Сбой датчика температуры	Временная	Тройное измерение с интервалом 10 секунд	Отфильтровывает случайные выбросы
Управление отоплением	Ошибка алгоритма	Программная	N-версионное программирование модуля расчёта	Независимые версии снижают риск системной ошибки
Управление замками	Отказ замка	Структурная	Механический резервный ключ	Позволяет открыть дверь при отказе электроники
Управление замками	Перехват сигнала	Информационная	Шифрование канала управления и контрольные суммы	Защищает от несанкционированного доступа
Управление замками	Отказ сети	Функциональная	Локальный контроллер с автономным режимом	Работает без облака
Удалённое управление	Отказ облачного сервиса	Структурная	Два независимых облачных провайдера	Переключение при недоступности одного
Удалённое управление	Потеря соединения	Временная	Автоматическое повторное подключение с интервалом	Восстанавливает связь без участия пользователя
Удалённое управление	Утечка данных	Информационная	Шифрование end-to-end и токены доступа	Защищает от перехвата
Удалённое управление	Отказ мобильного приложения	Программная	Две независимые версии приложения iOS и Android	Позволяет управлять с другого устройства